

Esempio con Kubernetes

(collocato su minikube) 2025-05-21

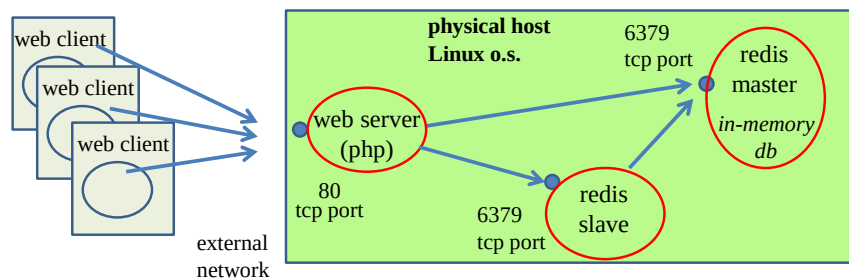
Tratto dal magnifico esempio: "Deploying PHP Guestbook application with Redis" (<https://kubernetes.io/docs/tutorials/stateless-application/guestbook/>)

L'esempio di applicazione da realizzare.

L'applicazione guestbook è formata da tre componenti, ciascuna delle quali è implementata in un container:

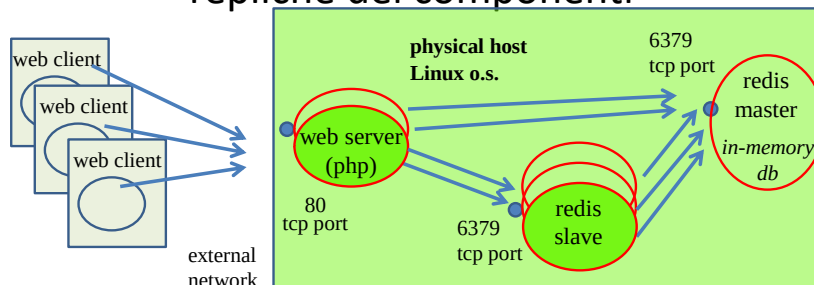
1. un componente redis-master che contiene il database redis in memoria e che può leggere e scrivere sul database.
2. un componente redis-slave che riceve ordini di lettura ma non li effettua direttamente, bensì si rivolge al componente readis-master.
3. il frontend, ovvero un web server implementato in php che riceve un messaggio dall'utente sul web browser. Nel messaggio l'utente passa una stringa da inserire nel database in memoria redis e vuole ricevere l'elenco delle stringhe presenti nel database redis. Il codice php ordina la scrittura del messaggio sul redis-master e poi ordina la lettura al redis-slave di tutte le stringhe del db e le visualizza all'utente.

Struttura dell'applicazione



Alcune componenti dell'applicazione possono essere replicate, per parallelizzare i compiti: queste componenti sono il web server e il readis-slave. Il redis-mster non può essere replicato.

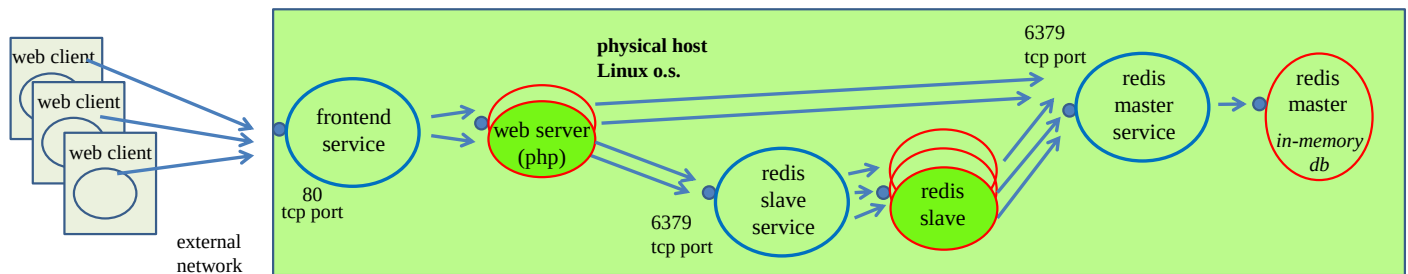
repliche dei componenti



Kubernetesizzare l'applicazione.

Ciascun componente viene realizzato da un pod. Per ciascun insieme di componenti analoghi si istanzia un deployment, e davanti a ciascun deployment si istanzia un service che si occupa di bilanciare il carico di richieste tra i diversi pods a cui fa da interfaccia.

strutturazione in deployments, pods e services dell'applicazione



Distinguiamo perciò:

- Deployment redis-master: uno solo.
- Service redis-master.
- Deployment redis-slave: più di uno.
- Service redis-slave.
- Deployment frontend: più di uno.
- Service frontend.

Aspetti di collegamento tra i pods:

1. Il bilanciamento di carico viene realizzato da kubernetes stesso.
2. Ciascun pod accede agli altri pods specificando, nel proprio codice, il nome del servizio a cui vuole accedere.
3. Il nome del servizio (redis-master, redis-slave) viene risolto automaticamente in un indirizzo IP dal DNS implicito fornito da kubernetes stesso. Perciò, nel codice php che realizza il web server troviamo una istruzione che cita il servizio redis-slave ed un'altra che cita il servizio redis-master.

Estratto da `php-redis/guestbook.php`

```
if ( una certa condizione )
    $host = 'redis-master';
    $client = new Predis\Client([
        'scheme' => 'tcp',
        'host'    => $host,
        'port'    => 6379,
```

```

    });
} else {
    $host = 'redis-slave';
    $client = new Predis\Client([
        'scheme' => 'tcp',
        'host'    => $host,
        'port'    => 6379,
    ]);
}

```

Analogamente, nella riga di comando con cui si lancia il redis-slave troviamo il nome del servizio redis-master.

Tratto da redis-slave/run.sh

```

else
    redis-server --slaveof redis-master 6379
fi

```

L'eseguibile redis-server, per svolgere il ruolo di slave, viene lanciato con un parametro che dice chi è il master a cui rivolgersi, tale master viene passato come fosse un nome di host () e viene risolto dal DNS implicito di kubernetes.

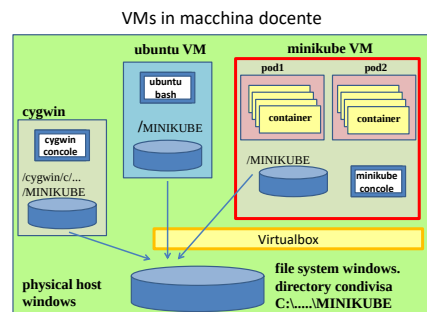
Altri sistemi analoghi a kubernetes non forniscono un servizio di DNS implicito, quindi occorre procedere con altri strumenti.

4. Il caso del servizio frontend è differente, poiché il servizio frontend viene acceduto dai browser esterni, cioè NON da container gestiti dall'esterno. Dall'esterno perciò non si può usufruire del servizio implicito del DNS di kubernetes. Dovremo ottenere da kubernetes l'indirizzo del frontend e farlo conoscere al browser.

Dispiegare l'applicazione.

L'ambiente delle prove.

Ricordo che nelle prove sfrutterò la strutturazione con 3 macchine virtuali che condividono una directory in cui c'è il codice. Suppongo di avere già creato le immagini dei container che lancerò, partendo dalla macchina virtuale ubuntu.



Si comincia andando nella shell cygwin, mettendo in esecuzione il cluster minikube e spostandosi nella directory principale di progetto.

```
minikube start  
cd ./examples-master/  
cd guestbook
```

Metto in esecuzione il pod per il redis-master, cioè dispiego il deployment per il redis-master.

```
kubectl apply -f redis-master-deployment.yaml
```

```
# redis-master-deployment.yaml  
  
apiVersion: apps/v1 # for versions before 1.9.0 use apps/v1beta2  
kind: Deployment  
metadata:  
  name: redis-master           #nome del servizio  
  labels:  
    app: redis  
spec:  
  selector:  
    matchLabels:  
      app: redis  
      role: master  
      tier: backend  
  replicas: 1  
  template:  
    metadata:  
      labels:  
        app: redis  
        role: master  
        tier: backend  
    spec:  
      containers:  
        - name: master  
          image: redis  
          resources:  
            requests:  
              cpu: 100m  
              memory: 100Mi  
          ports:  
            - containerPort: 6379
```

```
kubectl get pods  
kubectl logs -f redis-master-596696dd4-sqmxg      (è NOME PODS)
```

NB: in caso di errori, Se volessi eliminare il deployment userei:
kubectl delete redis-master-deployment.yaml

Creo il service per il redis-master

```
kubectl apply -f redis-master-service.yaml
```

```
# redis-master-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: redis-master
  labels:
    app: redis
    role: master
    tier: backend
spec:
  ports:
    - port: 6379
      targetPort: 6379
  selector:
    app: redis
    role: master
    tier: backend
```

```
kubectl get service
```

Metto in esecuzione i pods per i redis-slave, cioè dispiego il deployment per il redis-slave.

```
kubectl apply -f redis-slave-deployment.yaml
```

```
# redis-slave-deployment.yaml
apiVersion: apps/v1 # for versions before 1.9.0 use apps/v1beta2
kind: Deployment
metadata:
  name: redis-slave
  labels:
    app: redis
spec:
  selector:
    matchLabels:
      app: redis
      role: slave
      tier: backend
  replicas: 2
  template:
    metadata:
      labels:
        app: redis
        role: slave
        tier: backend
    spec:
```

```

containers:
- name: slave
  image: gcr.io/google_samples/gb-redisslave:v1
  resources:
    requests:
      cpu: 100m
      memory: 100Mi
env:
- name: GET_HOSTS_FROM
  value: dns
  # Using `GET_HOSTS_FROM=dns` requires your cluster to
  # provide a dns service. As of Kubernetes 1.3,
  # DNS is a built-in
  # service launched automatically. However,
  # if the cluster you are using
  # does not have a built-in DNS service, you can instead
  # access an environment variable to find the master
  # service's host. To do so, comment out the
  # 'value: dns' line above, and
  # uncomment the line below:
  # value: env
ports:
- containerPort: 6379

```

kubectl get pods

Creo il service per il redis-slave

kubectl apply -f redis-slave-service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: redis-slave
  labels:
    app: redis
    role: slave
    tier: backend
spec:
  ports:
  - port: 6379
  selector:
    app: redis
    role: slave
    tier: backend

```

kubectl get service

kubectl get pods

Metto in esecuzione i pods per i redis-slave, cioè dispiego il deployment per il frontend.

```
kubectl apply -f frontend-deployment.yaml
```

```
# frontend-deployment.yaml
apiVersion: apps/v1 # for versions before 1.9.0 use apps/v1beta2
kind: Deployment
metadata:
  name: frontend
  labels:
    app: guestbook
spec:
  selector:
    matchLabels:
      app: guestbook
      tier: frontend
  replicas: 3
  template:
    metadata:
      labels:
        app: guestbook
        tier: frontend
    spec:
      containers:
        - name: php-redis
          image: image: us-docker.pkg.dev/google-
samples/containers/gke/gb-frontent:v5
          resources:
            requests:
              cpu: 100m
              memory: 100Mi
          env:
            - name: GET_HOSTS_FROM
              value: dns
              # Using `GET_HOSTS_FROM=dns` requires your cluster to
              # provide a dns service. As of Kubernetes 1.3,
              # DNS is a built-in
              # service launched automatically. However, if
              # the cluster you are using
              # does not have a built-in DNS service, you can instead
              # access an environment variable to find the master
              # service's host. To do so, comment out the
              # 'value: dns' line above, and
              # uncomment the line below:
              # value: env
      ports:
        - containerPort: 80
```

```
kubectl get pods -l app=guestbook -l tier=frontend
```

Creo ed espongo il servizio frontend per l'applicazione.

```
kubectl apply -f frontend-service.yaml
```

```
# frontend-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: frontend
  labels:
    app: guestbook
    tier: frontend
spec:
  # comment or delete the following line if you
  # want to use a LoadBalancer
  type: NodePort
  # if your cluster supports it, uncomment the following
  # to automatically create
  # an external load-balanced IP for the frontend service.
  # type: LoadBalancer
  ports:
    - port: 80
  selector:
    app: guestbook
    tier: frontend
```

```
kubectl get services
```

Guarda il servizio di frontend richiedendone l'url di accesso. Verrà visualizzato l'URL con cui accedere al servizio. Copia l'URL per dopo.

```
minikube service frontend --url
```

```
kubectl get service frontend
```

Apri un browser, incolla l'URL di accesso nella barra degli indirizzi e fruisce dell'applicazione.

Aumenta il numero di repliche del frontend.

```
kubectl scale deployment frontend --replicas=5
```

```
kubectl get pods
```

Diminuisce il numero di repliche del frontend.

```
kubectl scale deployment frontend --replicas=2
```

```
kubectl get pods
```


Elimina i services ed i deployments.

```
kubectl delete deployment -l app=redis  
kubectl get pods  
kubectl delete service -l app=redis  
kubectl get pods  
kubectl delete deployment -l app=guestbook  
kubectl delete service -l app=guestbook  
kubectl get pods  
kubectl get services
```

Infine, ferma il cluster.

```
minikube stop
```